

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL PERSONALIZADO
(NIVEL 4 (INTEGRADOR))

Track 03: Sistema de Gestión con Base de Datos Relacional

«El Archivo Notarial y la Bóveda de Datos ACID»

Modelado de datos y persistencia robusta en Python: Conexión con SQLite y PostgreSQL, operaciones CRUD seguras con parámetros SQL, prevención de inyecciones y transacciones ACID.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Integrador / Producción | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Persistencia de Datos e Integridad Transaccional (ACID)	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Diagrama de Capas: Aplicación <-> Repositorio <-> Motor SQL	Pág. 5
Capítulo 3	Implementación Práctica en Python: Repositorio de Datos Seguro con SQLite	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Gestión de Bases de Datos	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender el modelo relacional de datos, las claves primarias/foráneas y la integridad transaccional ACID.



Competencia Práctica

Construir un sistema de persistencia completo con repositorios en Python puro interactuando con SQLite / PostgreSQL.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Persistencia de Datos e Integridad Transaccional (ACID)

La memoria RAM se borra al apagar la computadora; una base de datos relacional garantiza que la información de tus clientes y finanzas persista para siempre de forma atómica e íntegra.

Metáfora Central: El Archivo Notarial y la Bóveda de Datos ACID

Una base de datos relacional es como una bóveda notarial de alta seguridad: cada tabla es un libro de registros con columnas estrictas, y cada transacción es un contrato firmado. O se realizan todos los pasos de la operación o se cancela por completo sin dejar inconsistencias a medias.

Principios Teóricos y Modelo Mental

Propiedades ACID: Atomicidad (todo o nada), Consistencia (cumple reglas), Aislamiento (conurrencia segura), Durabilidad (persiste en disco).

Inyección SQL: La vulnerabilidad #1 en bases de datos; ocurre al concatenar texto crudo en queries. Se previene siempre con consultas parametrizadas (?) o (%s).

Regla de Oro en Python

NUNCA uses f-strings para construir sentencias SQL (ej: `f'SELECT * FROM u WHERE id={id}'`); usa siempre queries parametrizadas con tuplas.

2. Diagrama de Capas: Aplicación <-> Repositorio <-> Motor SQL

Arquitectura en capas (Layered Architecture) para aislar las sentencias SQL de la lógica de negocio.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	La capa de negocio solicita guardar o consultar una entidad.	Llamada a método del Repositorio
2. Evaluación	El Repositorio abre una conexión/cursor y prepara la sentencia parametrizada.	Preparación de la query
3. Transformación	El motor de base de datos ejecuta la transacción y valida claves únicas.	Ejecución ACID en disco
4. Retorno / Salida	Se realiza commit() para asegurar los cambios y se cierra la conexión de forma segura.	Datos persistidos permanentemente

Visualización Mental

Utiliza siempre context managers (with sqlite3.connect(...) as conn:) para asegurar el cierre de conexiones.

3. Repositorio de Datos Seguro con SQLite

Implementación de persistencia relacional con transacciones y consultas parametrizadas:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
import sqlite3

class RepositorioUsuarios:
    def __init__(self, db_path: str = "app.db"):
        self.db_path = db_path
        self._crear_tabla()

    def _crear_tabla(self) -> None:
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            cursor.execute('''
                CREATE TABLE IF NOT EXISTS usuarios (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nombre TEXT NOT NULL,
                    email TEXT UNIQUE NOT NULL,
                    saldo REAL DEFAULT 0.0
                )
            ''')
            conn.commit()

    def insertar_usuario(self, nombre: str, email: str, saldo: float) -> int:
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            # Consulta parametrizada segura contra Inyección SQL
            cursor.execute(
                "INSERT INTO usuarios (nombre, email, saldo) VALUES (?, ?, ?)",
                (nombre, email, saldo)
            )
            conn.commit()
            return cursor.lastrowid
```

Análisis del Código Fuente

Clase Repository que encapsula la lógica SQL, maneja el ciclo de vida de conexiones y previene vulnerabilidades de inyección SQL.

4. Gotchas en Gestión de Bases de Datos

Errores críticos que provocan pérdida de datos o brechas de seguridad:

⚠ Gotcha Frecuente (Trampa de Principiante)

Concatenar variables de usuario directamente dentro de sentencias SQL, permitiendo ataques de Inyección SQL.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
cursor.execute(f"SELECT * FROM users WHERE user = '{user}'") # ¡Vulnerable!
```

✓ Patrón Pythonic / Correcto

```
cursor.execute("SELECT * FROM users WHERE user = ?", (user,)) # Inmune a inyección
```

🛡 Consejo de Resiliencia en Producción

Crea siempre índices (CREATE INDEX) sobre las columnas que uses frecuentemente en cláusulas WHERE o JOIN.

5. Resumen Ejecutivo y Conclusiones

¡Felicitaciones! Has dominado el diseño y persistencia de bases de datos relacionales en Python.



Logro Alcanzado en esta Clase

Capacidad para construir sistemas de información profesionales con integridad de datos garantizada.

Notas del Instructor y Sigüientes Pasos

Presenta este sistema con su esquema relacional como parte de tu proyecto final integrador.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Implementa una transacción bancaria que transfiera saldo entre dos usuarios asegurando atomicidad con rollback en caso de error.